



**Czech
Technical
University
in Prague**

F3

**Faculty of Electrical Engineering
Department of Cybernetics**

Technical Report

Robotics – B3B33ROB1

**Václav Beran
Tomáš Janoušek
Mathias Palme**

20. 1. 2026

Contents

Introduction	1
1 Problem analysis	3
1.1 Maze localization	3
1.2 Coordinate systems	3
1.3 Collision-free movement	3
2 Solution	4
2.1 Localization	4
2.1.1 Orientation Representation	4
2.2 Camera-to-Robot Coordinate Transformation	5
2.2.1 Calibration Data Acquisition	5
2.2.2 Image Processing and Feature Extraction	5
2.2.3 Estimation of the Camera-to-Robot Transformation Matrix	6
2.2.4 Perspective Projection Model	6
2.2.5 Pose Estimation via PnP	6
2.2.6 Camera-to-Robot Transformation	7
2.3 SE(3) construction for Hoop IK	7
2.3.1 Orthonormal Basis Construction	7
2.3.2 Tool Offset Translation	8
3. Homogeneous Transformation Matrix	8
2.3.3 Required Constraints	8
2.4 Path Planning and Trajectory Generation	8
2.4.1 Vision-Based Pose Alignment	9
2.4.2 Coordinate Frame Transformation	9
2.4.3 Configuration Selection and Inverse Kinematics	9
2.4.4 Arm Vector Decision	9
2.4.5 Approach and Execution Strategy	10
3 Implementation	11
3.1 Camera module	11
3.2 Motion generation	11
3.3 Robot control	11
4 Conclusion	12
Bibliography	13

Figures

1 All maze variations	1
2 CRS97 work station	2

Tables

2.1 Hough Circle Transform parameters used for calibration target detection	5
---	---

Introduction

This report presents the final project developed within the course B3B33ROB1 – Robotics at the Faculty of Electrical Engineering, Czech Technical University in Prague. The project was implemented on the CRS97 2) robotic workstation and focused on an autonomous solution of three predefined mazes (variants A, B, and C, as seen in figure 1) by guiding the hoop attached to the robot's end-effector around the mazes without touching them. The maze layouts were known in advance, allowing the task to be formulated primarily as a trajectory planning and execution problem. The solution required correct handling of coordinate transformations, generation of feasible end-effector trajectories, and safe execution of motion on real robotic hardware. The objective of the project was to design, implement, and verify a control program capable of reliably solving all maze variations, and this report describes the adopted approach and evaluates the achieved results.

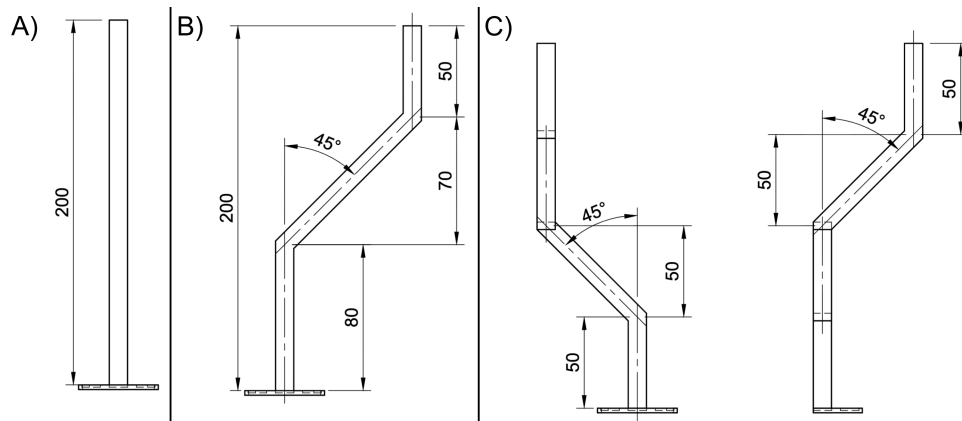


Figure 1: All maze variations

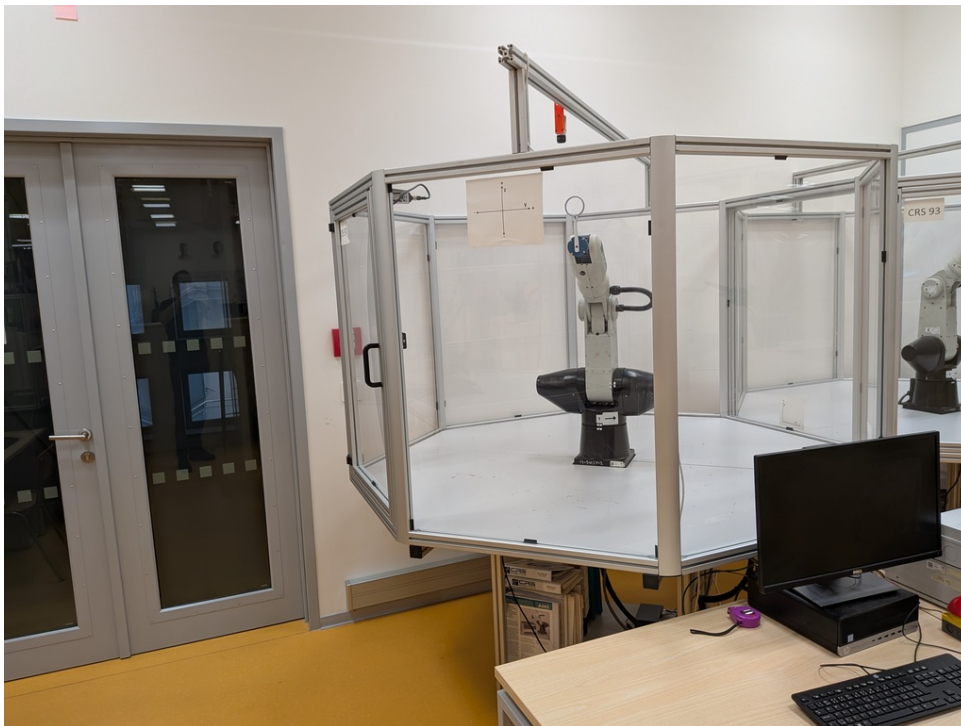


Figure 2: CRS97 work station

Chapter 1

Problem analysis

The autonomous solution of the maze task required addressing several key technical problems.

1.1 Maze localization

Since the position and orientation of the maze were not fixed with respect to the robot, it was first necessary to perform maze localization using computer vision methods based on ArUco fiducial markers, allowing the maze reference frame to be identified in the camera coordinate system.

1.2 Coordinate systems

The detected maze position had to be then transformed from the camera coordinate system to the robot coordinate system, which required accurate calibration.

1.3 Collision-free movement

Although the inverse kinematics of the CRS97 robot end-effector were provided by the control system[1], the task additionally required solving the inverse kinematics of the hoop, imposing physical constraints on the allowable end-effector poses during maze traversal. Finally, a suitable path selection strategy had to be defined to ensure collision-free motion of the hoop through the mazes while respecting the robot's physical limitations.

Chapter 2

Solution

2.1 Localization

The computer vision subsystem provides reliable localization of the maze within the camera coordinate frame using ArUco fiducial markers. A Basler RGB camera is employed, with intrinsic parameters and lens distortion coefficients obtained offline and loaded at runtime. Captured images are converted to grayscale and processed using the OpenCV ArUco module to detect markers from a predefined dictionary.

For each detected marker, its three-dimensional pose relative to the camera is estimated using the Perspective- n -Point (PnP) method, yielding a rotation vector $\mathbf{r}$ and a translation vector $\mathbf{t}$ that define the transformation from the marker frame to the camera frame. The maze position is estimated as the average translation of selected reference markers, ensuring a consistent maze reference frame despite placement variations.

2.1.1 Orientation Representation

The rotation vector is converted to a rotation matrix $\mathbf{R}$, from which the maze orientation is derived. The rotation matrix is defined as

$$\mathbf{R} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}.$$

To facilitate interpretation, $\mathbf{R}$ is converted to Euler angles. An auxiliary term used in this conversion is

$$s_y = \sqrt{r_{11}^2 + r_{21}^2}.$$

This localization framework forms the basis for subsequent coordinate transformations and motion planning.

2.2 Camera-to-Robot Coordinate Transformation

This chapter describes the calibration procedure used to estimate the homogeneous transformation matrix $\mathbf{T}_{RC}$, which maps points from the camera coordinate system to the robot coordinate system.

2.2.1 Calibration Data Acquisition

Calibration is performed using $N = 12$ reference points sampled within the robot workspace. The points are uniformly distributed within the following bounds:

$$x \in [0.35, 0.55], \quad y \in [-0.22, 0.22], \quad z \in [0.10, 0.40].$$

At each reference position, an image of a circular calibration target (hoop) is captured by the camera, establishing 3D–2D correspondences between robot coordinates and image measurements.

2.2.2 Image Processing and Feature Extraction

The center of the calibration target is detected in the image using the Hough Circle Transform applied to a preprocessed grayscale image. The detected circle center provides the pixel coordinates associated with each calibration point and is used for estimating the transformation matrix $\mathbf{T}_{RC}$.

Hough Circle Detection Parameters

Table 2.1 summarizes the parameters used for the Hough Circle Transform.

Parameter	Value
Detection method	HOUGH_GRADIENT
Inverse ratio (dp)	1.2
Minimum center distance	50 px
Canny high threshold ($param1$)	100
Accumulator threshold ($param2$)	30
Minimum radius	5 px
Maximum radius	Unconstrained

Table 2.1: Hough Circle Transform parameters used for calibration target detection

The resulting set of 3D robot points and corresponding 2D image coordinates is used to compute the camera-to-robot transformation matrix $\mathbf{T}_{RC}$.

2.2.3 Estimation of the Camera-to-Robot Transformation Matrix

The camera-to-robot coordinate transformation matrix $\mathbf{T}_{RC}$ is estimated using a set of known 3D points expressed in the robot coordinate system and their corresponding 2D projections in the camera image. Let

$$\mathbf{P}_R^{(i)} = \begin{bmatrix} X_i & Y_i & Z_i \end{bmatrix}^T$$

denote the i -th calibration point in the robot coordinate frame, and

$$\mathbf{p}^{(i)} = \begin{bmatrix} u_i & v_i \end{bmatrix}^T$$

it's observed pixel coordinates in the image.

2.2.4 Perspective Projection Model

The relationship between a 3D point in the robot frame and its image projection is described by the pinhole camera model

$$s \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} = \mathbf{K} \left(\mathbf{R}_{CR} \mathbf{P}_R^{(i)} + \mathbf{t}_{CR} \right),$$

where s is a projective scale factor, $\mathbf{K}$ is the intrinsic camera matrix, and $(\mathbf{R}_{CR}, \mathbf{t}_{CR})$ represent the rotation and translation from the robot frame to the camera frame. Lens distortion is modeled using previously calibrated distortion coefficients.

2.2.5 Pose Estimation via PnP

The unknown extrinsic parameters $(\mathbf{R}_{CR}, \mathbf{t}_{CR})$ are estimated using a Perspective- n -Point (PnP) formulation. A robust solution is obtained by minimizing the reprojection error

$$\sum_i \left\| \mathbf{p}^{(i)} - \pi \left(\mathbf{K} \left(\mathbf{R}_{CR} \mathbf{P}_R^{(i)} + \mathbf{t}_{CR} \right) \right) \right\|^2,$$

where $\pi(\cdot)$ denotes the perspective projection from homogeneous camera coordinates to image coordinates.

To improve robustness against outliers, the estimation is performed using a RANSAC-based PnP solver. The SQPnP algorithm is employed as the underlying pose estimation method. A maximum reprojection error threshold of 8.0 pixels is used to classify inliers, and the algorithm is allowed up to 400 RANSAC iterations. After inlier selection, the pose is refined using nonlinear optimization over the inlier set.

The estimated rotation is represented internally as a rotation vector $\mathbf{r}$ and converted to a rotation matrix $\mathbf{R}_{CR}$ using the Rodrigues formula.

2.2.6 Camera-to-Robot Transformation

The PnP solution provides the transformation from the robot frame to the camera frame. To obtain the desired camera-to-robot transformation, the inverse rigid-body transform is computed as

$$\mathbf{R}_{RC} = \mathbf{R}_{CR}^T, \quad \mathbf{t}_{RC} = -\mathbf{R}_{CR}^T \mathbf{t}_{CR}.$$

The resulting homogeneous transformation matrix is then given by

$$\mathbf{T}_{RC} = \begin{bmatrix} \mathbf{R}_{RC} & \mathbf{t}_{RC} \\ \mathbf{0}^T & 1 \end{bmatrix},$$

which enables direct transformation of points from the camera coordinate system to the robot coordinate system. Specifically, a point expressed in camera coordinates

$$\mathbf{P}_C = [x_C \ y_C \ z_C]^T$$

is first written in homogeneous form as

$$\mathbf{P}_{C,h} = [x_C \ y_C \ z_C \ 1]^T.$$

The corresponding point in the robot coordinate system is then obtained by

$$\mathbf{P}_{R,h} = \mathbf{T}_{RC} \mathbf{P}_{C,h},$$

where $\mathbf{T}_{RC}$ is the estimated homogeneous camera-to-robot transformation matrix. The Cartesian robot coordinates are recovered as

$$\mathbf{P}_R = [X_R \ Y_R \ Z_R]^T,$$

given by the first three components of $\mathbf{P}_{R,h}$. If the translation component of $\mathbf{T}_{RC}$ is expressed in millimeters, the resulting coordinates are scaled accordingly to obtain metric units in meters.

2.3 $SE(3)$ construction for Hoop IK

In this chapter we construct a homogeneous transformation matrix $T \in SE(3)$ to define the robot's pose relative to a target position $\mathbf{P}_{circle}$, using a normal vector $\mathbf{v}_n$, which defines the plane of the Hoop, and an arm vector $\mathbf{v}_a$, that represents the direction of the Hoops arm..

2.3.1 Orthonormal Basis Construction

The rotation component is built by establishing three mutually orthogonal unit vectors:

- **Z-axis (Approach):** Defined as the normalized inverse of the target normal vector:

$$\hat{\mathbf{z}} = \frac{-\mathbf{v}_n}{\|\mathbf{v}_n\|}$$

- **X-axis (Alignment):** Defined by normalizing the arm vector:

$$\hat{\mathbf{x}} = \frac{\mathbf{v}_a}{\|\mathbf{v}_a\|}$$

- **Y-axis (Orthogonal):** Calculated via the cross product to complete the right-handed coordinate system:

$$\hat{\mathbf{y}} = \hat{\mathbf{z}} \times \hat{\mathbf{x}}$$

2.3.2 Tool Offset Translation

The final robot position $\mathbf{P}_{robot}$ is calculated by offsetting the target center $\mathbf{P}_{circle}$ along the alignment axis $\hat{\mathbf{x}}$ by the tool length L :

$$\mathbf{P}_{robot} = \mathbf{P}_{circle} + (\hat{\mathbf{x}} \cdot L)$$

3. Homogeneous Transformation Matrix

The resulting 4×4 matrix T is assembled as follows:

$$T = \left[\begin{array}{ccc|c} \hat{x}_x & \hat{y}_x & \hat{z}_x & P_{robot,x} \\ \hat{x}_y & \hat{y}_y & \hat{z}_y & P_{robot,y} \\ \hat{x}_z & \hat{y}_z & \hat{z}_z & P_{robot,z} \\ 0 & 0 & 0 & 1 \end{array} \right]$$

2.3.3 Required Constraints

For the matrix to be geometrically valid, the following conditions must be met:

1. **Orthogonality:** $\mathbf{v}_a \cdot \mathbf{v}_n \approx 0$
2. **Non-zero Magnitude:** $\|\mathbf{v}_a\| \neq 0$ and $\|\mathbf{v}_n\| \neq 0$

2.4 Path Planning and Trajectory Generation

The path planning process transforms an abstract set of puzzle points into a sequence of executable robot configurations. This involves three primary stages: spatial alignment via vision, coordinate frame mapping, and configuration space optimization.

2.4.1 Vision-Based Pose Alignment

The trajectory begins in the puzzle's local coordinate system. To align the trajectory with the physical object, the system detects ArUco markers to determine the puzzle's midpoint $\mathbf{P}_{mid}$ and orientation matrix R_{obs} . The raw trajectory points $\mathbf{P}_{local}$ are transformed into the camera frame $\mathbf{P}_{cam}$ using:

$$\mathbf{P}_{cam} = \mathbf{P}_{local} \cdot R_{obs} + (\mathbf{P}_{mid} - \delta_z) \quad (2.1)$$

where δ_z represents a vertical offset (e.g., 35mm) to account for the physical thickness of the puzzle mat, ensuring the tool center point (TCP) is correctly positioned relative to the depth of the maze.

2.4.2 Coordinate Frame Transformation

To execute the motion, points must be mapped from the camera frame to the robot's base frame. Using the calibrated homogeneous transformation matrix T_{RC} , each point $\mathbf{x} \in \mathbf{P}_{cam}$ is converted to the robot frame $\mathbf{x}_{rob}$ via homogeneous multiplication:

$$\begin{bmatrix} \mathbf{x}_{rob} \\ 1 \end{bmatrix} = T_{RC} \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} \quad (2.2)$$

2.4.3 Configuration Selection and Inverse Kinematics

For each Cartesian target $\mathbf{x}_{rob}$, the robot must find a joint configuration $\mathbf{q}$. Since robots are often redundant, the Inverse Kinematics (IK) function returns a set of valid solutions $\mathcal{Q} = \{\mathbf{q}_1, \mathbf{q}_2, \dots, \mathbf{q}_n\}$.

To ensure smooth motion and avoid unnecessary joint flips, the planner selects the configuration $\mathbf{q}^*$ that minimizes the Euclidean distance to the current configuration $\mathbf{q}_{curr}$:

$$\mathbf{q}^* = \arg \min_{\mathbf{q} \in \mathcal{Q}} \|\mathbf{q} - \mathbf{q}_{curr}\|_2 \quad (2.3)$$

Subject to the joint limit constraint:

$$\mathbf{q}_{min} \leq \mathbf{q}^* \leq \mathbf{q}_{max} \quad (2.4)$$

2.4.4 Arm Vector Decision

To prevent kinematic singularities or collisions, the system evaluates multiple approach directions (arm vectors $\mathbf{v}_a$). A vector is deemed valid only if a continuous chain of IK solutions exists for the entire trajectory:

$$\forall \mathbf{x} \in \text{Trajectory}, \quad \text{IK}(\text{Pose}(\mathbf{x}, \mathbf{v}_a, \hat{\mathbf{n}})) \neq \emptyset \quad (2.5)$$

In our implementation, we tested 8 different arm vectors, each offset by 45° . Both the number of arm vectors and the angular offset are configurable parameters, allowing the system to adapt to different robot geometries or

workspace constraints. The vector with the highest x-component was then selected as the arm vector. This is done to minimize the danger of the robot's body hitting the maze.

2.4.5 Approach and Execution Strategy

The execution is structured to ensure safety and repeatability using a four-phase trajectory:

1. **Approach Phase:** A safe start pose $\mathbf{P}_{start}$ is generated by offsetting the first trajectory point along the normal vector $\hat{\mathbf{n}}$:

$$\mathbf{P}_{start} = \mathbf{P}_{rob,0} + 0.05 \cdot \hat{\mathbf{n}} \quad (2.6)$$

2. **Forward Execution:** The robot iterates through the sequence of poses T_i generated by the Pose Composer, moving linearly between discrete points.
3. **Reverse Execution:** To return to the safety zone, the sequence is traversed in reverse order: $\mathcal{T}_{rev} = \{T_n, T_{n-1}, \dots, T_0\}$.
4. **Retraction:** The robot returns to $\mathbf{P}_{start}$ and finally to a predefined *soft-home* position.

Chapter 3

Implementation

The implementation is structured as a modular software system in which individual components are responsible for perception, coordinate transformation, motion generation, and robot control.

3.1 Camera module

The camera module performs image acquisition and computer vision processing, including ArUco marker detection and estimation of the maze pose in the camera coordinate frame. The resulting spatial information is passed to the coordinate transformation module, which converts maze-related points from the camera frame to the robot base frame using a calibrated camera-robot transformation.

3.2 Motion generation

Maze geometry and predefined trajectories are handled by the maze representation module, which provides an ordered sequence of waypoints describing the desired path through the maze. These waypoints serve as inputs to the pose generation module, which computes feasible $SE(3)$ end-effector poses by applying the hoop inverse kinematics and enforcing orientation constraints derived from the maze geometry.

3.3 Robot control

The path validation and selection module evaluates candidate arm orientations and verifies kinematic feasibility by repeatedly invoking the robot inverse kinematics solver and checking joint limits. Once a valid pose sequence is selected, the motion execution module translates the poses into joint configurations, selects the configuration closest to the previous state to ensure smooth motion, and commands the robot to execute the trajectory. All modules interact through clearly defined interfaces, enabling reliable autonomous operation of the CRS97 robot and simplifying debugging and future extensions of the system.



Chapter 4

Conclusion

This project demonstrates the successful integration of computer vision, coordinate transformation, inverse kinematics, and motion planning for autonomous maze solving using the CRS97 robotic workstation. The implemented solution reliably localizes the maze using ArUco markers, transforms the detected geometry into the robot coordinate system, and generates feasible end-effector trajectories that respect the physical constraints imposed by the robot's structure. The modular architecture enables clear separation of individual components and supports robust execution on real hardware. The results confirm that the implemented approach is capable of solving all assigned maze variants and provides a solid foundation for further extensions, such as solutions of more complicated maze types. The ArUco detection system has all the disadvantages of an optical detection system and is heavily reliant on good lighting and minimal obstruction.



Bibliography

- [1] *CRS93 and CRS97 stations*. Accessed: 2025-01-04. Robotics Course: Robot Stations Documentation, 2025. URL: <https://robotics-robot-stations.readthedocs.io/en/latest/stations/crs.html>.